

DEVELOPER HANDBOOK

codescribe

A strictly-local, fine-tuned + retrieval-augmented + neuro-symbolic coding-assistant stack — the complete developer's guide.

Scope: everything an engineer needs to understand and build this stack — background, concept refreshers (with a deep dive on RAG and neuro-symbolic AI), architecture, the code map, the two-machine setup, the evaluation results, and the landmines that cost real debugging time.

What this is: a coding assistant for a *private codebase* — one you cannot send to a cloud model (NDA, customer contract, regulatory or security constraint). It fine-tunes a small open-weights coder on your code, serves it locally, and augments it with a retrieval index and a symbolic knowledge graph, so a modest model that *knows your codebase* beats a big model that has never seen it.

This is the teachable, source-agnostic reference. It names no specific codebase, host, or operator — everything is described by role and by concept, so it is safe to share.

Contents

Part I — Orientation

- 1.1 What this stack is (and is not)
- 1.2 The prime directive: strictly local
- 1.3 The capability ceiling
- 1.4 The mental model: four phases + two memories

Part II — Concept refreshers

- 2.1 Language models & the base model
- 2.2 Fine-tuning: LoRA → QLoRA → Unsloth
- 2.3 Model collapse & why we eval-gate
- 2.4 RAG — retrieval-augmented generation (deep dive)
- 2.5 Neuro-symbolic AI (deep dive)
- 2.6 MCP — how tools reach the model

Part III — Architecture & code map

- 3.1 The end-to-end pipeline
- 3.2 Package by package
- 3.3 The two data stores
- 3.4 Configuration

Part IV — Infrastructure & operations

- 4.1 The two machine roles
- 4.2 Runbooks
- 4.3 The strictly-local posture in practice

Part V — Evaluation: does each layer earn its place?

- 5.1 Methodology
- 5.2 Single-hop results
- 5.3 Multi-hop results — the neuro-symbolic win

Part VI — Landmines: the traps that already bit

Part VII — Current state & roadmap

Appendix A — Glossary

Appendix B — Package & command index

Part I — Orientation

1.1 What this stack is (and is not)

codescribe turns a small, open-weights code model into a competent assistant *for one specific private codebase* — and does the whole thing on your own hardware, with nothing leaving the machine.

You have a private codebase: tens of thousands of files, years of history, lots of project-specific patterns. You want an assistant that *actually knows your code* — names your modules correctly, follows your idioms, knows where the bodies are buried — and you cannot send any of it to a cloud model. There are three ways to give a generic LLM project-specific knowledge: prompting (doesn't scale), **RAG** (retrieve relevant snippets at query time), and **fine-tuning** (bake knowledge into the weights). This stack does **fine-tuning + RAG together**, and adds a third, sharper layer — a **neuro-symbolic** knowledge graph with a deductive reasoner.

Concretely, it is a pipeline that: (1) turns any source tree into training data; (2) fine-tunes **Qwen 2.5 Coder 7B** with QLoRA on an 8 GB laptop GPU; (3) serves the result through a local OpenAI-compatible inference server; and (4) lets you drive any coding harness against it. On top sit the two memories — a RAG index (fuzzy recall of code, docs, issues, changes) and the neuro-symbolic layer (exact, provenance-backed structural facts + reasoning).

It is **not** a general chat assistant, not a hosted product, and not trying to beat frontier models. It is a single-operator system tuned to one codebase.

1.2 The prime directive: strictly local

Every design decision bows to one constraint: **data leaving the host is a failure mode, not a preference**. This is the reason the stack exists. Concrete consequences:

- **No cloud GPUs** — training and inference run on your own hardware.
- **No cloud embedding APIs** — embeddings for RAG run locally on the same GPU.
- **No cloud vector stores** — SQLite + `sqlite-vec` is the entire vector layer.
- **No telemetry** — no Weights & Biases, no Sentry, no model-hub upload, no `POST` to anything off an explicit allowlist (the model hub for *downloads*; your own Perforce and Bugzilla servers on the LAN).
- **Loopback by default** — any HTTP component binds `127.0.0.1`; binding `0.0.0.0` needs an explicit `unsafe_bind_all=True` flag and trips static-test warnings.

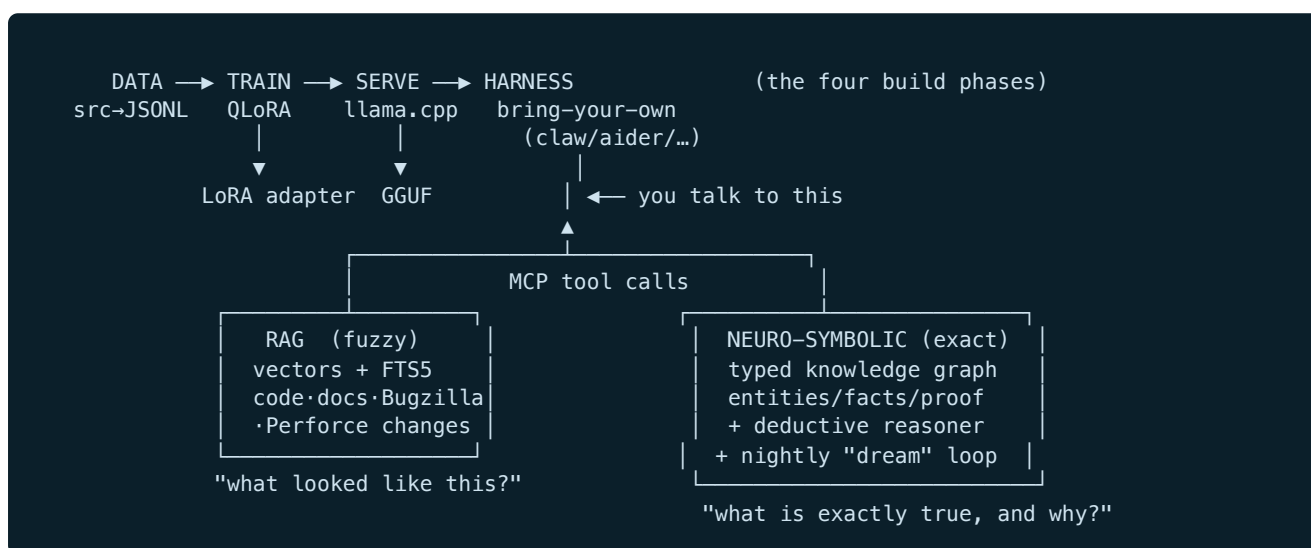
The whole system's Definition-of-Done is verified with an **egress audit**: a full session run under `strace -e trace=connect` showing *zero non-loopback* `connect()` calls. Any feature that touches the network must re-pass it.

Rule you will not talk your way out of If a task seems to need a cloud call, an upload, or a public endpoint — **stop**. The answer is "do it locally or don't do it." This has already killed otherwise-reasonable ideas, and that is the point.

1.3 The capability ceiling

The accepted ceiling is **intern-level** assistance, and that is fine. A 7B model fine-tuned on one codebase will not reason like a frontier model. The bet: *a modest model that knows your code cold, cites real sources, and can reason over its structure* beats a bigger model that has never seen your code — for the narrow, high-frequency tasks (where is X defined, what calls Y, what breaks if I change Z, what's the convention here). Improvement comes from better base models, recipes, or local hardware — never the cloud.

1.4 The mental model: four phases + two memories



- **The four phases** depend in strict order: `data → train → serve → harness`. Each hides behind an interface so it can be swapped.
- **The two memories** are augmentations on top, both reached through **MCP tools**:
 - **RAG** — a *non-parametric, fuzzy* memory: "show me code/text that looks like this query." Fresh, broad, shallow about structure.
 - **Neuro-symbolic (NS)** — an *exact, structured* memory: "this function calls that one, defined here, and here is the proof." Precise, provenance-backed, can *reason* multi-hop.

The evaluation's headline (Part V) is that these two win on *different axes* and should be turned on for different jobs. NS is not "better RAG" — they are complementary.

Part II — Concept refreshers

Assume a strong engineer, rusty on the ML specifics. Skim what you know; the RAG (2.4) and neuro-symbolic (2.5) sections are the ones the rest of the stack leans on hardest.

2.1 Language models & the base model

An LLM predicts the next *token* given the preceding ones. Everything it "knows" is compressed into its weights — its **parametric memory**, which is (a) *frozen* at training time, (b) *lossy* (statistical tendencies, not verbatim facts — hence hallucination), and (c) *uncited*. RAG and NS exist to patch exactly these three weaknesses.

The base model is **Qwen 2.5 Coder 7B Instruct**, chosen for its strong code performance at a size that fine-tunes on 8 GB. Its tokenizer — including its **FIM** (fill-in-the-middle) tokens — is a *contract*: you cannot swap the base model without re-running the data pipeline, because the special tokens change.

2.2 Fine-tuning: LoRA → QLoRA → Unsloth

Fine-tuning continues training on your data so the weights absorb your domain. Full fine-tuning of a 7B needs far more VRAM than a laptop has; three stacked optimisations make it fit:

1. **LoRA (Low-Rank Adaptation)** — freeze the big matrices, learn small low-rank *adapters* alongside (~0.1–1% as many parameters). The output is a small, swappable adapter file, not a whole model.
2. **QLoRA (Quantized LoRA)** — additionally store the frozen base in **4-bit** (NF4), ~4× less memory; adapters still train in bf16. This is what makes a 7B trainable on 8 GB.
3. **Unsloth** — hand-optimised kernels for exactly this QLoRA path; faster and lower-memory than vanilla PEFT. Mandatory on the 8 GB card. Optimizer: `paged_adamw_8bit`.

The served model is exported to **Q4_K_M GGUF** — a 4-bit format the vendored `llama.cpp` server loads. Training samples come in three shapes (*FIM* completion, *document* language-modelling, *diff-instruction*), and splits are made **by file path, never by chunk**, so no chunk of a file leaks across train/val/test.

2.3 Model collapse & why we eval-gate

Train a model on its own synthetic output repeatedly and quality decays — variance collapses, rare patterns vanish. This is **model collapse** ("the curse of recursion"), a live risk because the neuro-symbolic layer *generates* synthetic training data. The defence is an **eval-gated retrain**, the only weight-touching phase: assemble real + capped-synthetic data, retrain, then a **beat-or-discard gate** requires the new adapter to beat the current one on a frozen, synthetic-free eval, or it is thrown away. Never promote a model that did not measurably beat its predecessor.

2.4 RAG — retrieval-augmented generation (deep dive)

RAG gives the model a second memory it can look things up in at inference time: an external, updatable index you *retrieve* from and *prepend* to the prompt. Instead of hoping the answer is

baked into the weights, you fetch relevant text and let the model read it.

```
query → [retrieve top-k relevant chunks] → prepend to prompt → model reads them → answer
      |
      two retrieval signals, fused:
      (1) semantic (embeddings + vector search)
      (2) lexical  (FTS5 keyword / BM25)
```

Embeddings & vector search

An **embedding** is a dense vector (here, **1024 floats**, from a local **bge-large** embedder) placing semantically similar texts near each other. Retrieve by embedding the query and finding the nearest index vectors (cosine/L2). This catches *meaning* — "authenticate a user" can match code that never says "authenticate." The vector store is **sqlite-vec**: `vec0` virtual tables holding `FLOAT[1024]`, KNN inside SQLite — the whole index is one `.db` file, no separate service.

Lexical search (FTS5) and why you still need it

Embeddings blur *exact* tokens — a specific function name, an error string, a flag. **FTS5** is SQLite's full-text search (inverted index, BM25, porter stemming); it nails exact-identifier lookups embeddings miss. Running both and fusing them is **hybrid retrieval**.

RRF — fusing the two rankings

Reciprocal Rank Fusion merges the two ranked lists without comparable scores: each document scores $\sum 1/(k + \text{rank}_i)$ (small constant k) and you re-sort. High rank by *either* signal floats a document up; high by *both* wins. Simple, robust, scale-agnostic.

What is indexed, and chunking

The index is not just code — it ingests several forms of institutional memory:

Source	Why it matters
Code files (chunked)	the actual implementation
Docs (Markdown)	design intent, READMEs
Bugzilla bugs	"why did we do X", failure history
Perforce changes / reviews	rationale, discussion, diffs

Chunking splits long files into retrievable units (functions, sections, windows) — you cannot embed a 2,000-line file as one vector meaningfully. Each chunk keeps its `file_path` + line span so a citation can be produced.

Landmine — external-content FTS5 The FTS5 tables mirror another table (`content='code_chunks'`) rather than storing their own copy. Deleting rows has a strict **order**: delete the *FTS* row **first** (while the content row still exists, so FTS recomputes the right terms to remove), *then* the vector row, *then* the content row. Wrong order leaves "ghost" terms that corrupt search. (See Part VI.)

When RAG vs fine-tuning vs tools vs NS

Use...	When you need...	Weakness
Fine-tune	fluent house style, broad structure, zero-latency recall	ages; invents specifics; can't cite
RAG	fresh, specific snippets; things newer than the fine-tune	shallow on structure; can't chain; retrieval latency
Tools (grep/etc.)	exact current file contents on demand	needs the agent to know what to look for
Neuro-symbolic	exact relationships, provenance, multi-hop reasoning	silent outside the graph; build cost

RAG's signature axis is freshness. It is measurably the *wrong* tool for structural reasoning — the multi-hop result in Part V shows it actively hurting.

2.5 Neuro-symbolic AI (deep dive)

Where the LLM is a fuzzy pattern-matcher and RAG is fuzzy lookup, the neuro-symbolic layer is an *exact*, provenance-backed knowledge graph with a *deductive reasoner*. "Neuro-symbolic" = the neural model (fuzzy, fluent) working with a symbolic system (precise, logical), each covering the other's blind spot.

Why bother, given the LLM and RAG?

1. **Be exact about structure.** "Does **A** call **B**?" has a true/false answer. A statistical model guesses; a graph *knows*.
2. **Cite real provenance.** Every symbolic fact carries the exact `file:line` it came from, so the model can relay a clickable citation.
3. **Reason multi-hop.** "Does **A** transitively call **C** through something?" needs *chaining*. LLMs are unreliable at it; RAG cannot do it at all (its chunks don't contain the connecting hop). A deductive engine chains by construction.

The knowledge graph

A typed graph stored as three core tables:

- **entities** — nodes with a `kind` (`file` | `symbol` | `module` | `repo` | `convention` | `decision` | `bug`), a canonical `name` , a `path` , and a JSON `meta` blob.
- **facts** — edges as triples (*subject*) —*predicate*→ (*object*). Predicates: `calls` , `imports` , `defined-in` , `depends-on` , `supersedes` , `convention-applies-to` , `decision-affects` . Each carries a **confidence** and a **validity window** (`valid_from` / `valid_to`).
- **provenance** — one-to-many *evidence* per fact (`source_kind` , `source_ref` like `pkg/module.py:142`); multiple sources *raise* confidence.

Two design choices matter enormously:

- **Deterministic backbone vs. fuzzy distillation.** Facts from deterministic AST/regex parsers get **confidence 1.0**; facts *distilled by an LLM* from messy interaction logs get **< 1.0**. You can always tell a

proven fact from a guessed one.

- **Temporal / soft-supersession.** Facts are not hard-deleted on change; they get a `valid_to` timestamp, so the graph keeps history and you can query "what was true as of date D."

The "dream" loop — nightly consolidation

Borrowing from how sleep consolidates memory, a nightly cycle runs five stages:

```
ingest → extract → reflect → synthesize → [retrain]
(raw    (facts    (dedup,   (reasoning-   (only if the
episodic from code resolve grounded training   eval gate
capture) + distill conflicts, training passes)
          from logs) raise conf) samples)
```

Raw interactions land in an episodes table; extraction turns them plus fresh code into facts; reflection deduplicates and resolves conflicts (bumping confidence when sources agree, superseding when they don't); synthesis produces reasoning-grounded training data; and — only if the eval gate passes — a retrain may run. Lineage tables record which cycles fed which weights, so a regression can be bisected.

The deductive reasoner — the intellectual core

A small, dependency-free, **read-only** backward-chaining engine over the graph. It derives *new* facts by transitive closure, each with a full proof trace. The rules are kept as *data*, Datalog-shaped and guaranteed to terminate:

```
reaches(A, B) :- calls(A, B).
reaches(A, C) :- calls(A, B), reaches(B, C).      # transitive call closure
depends(A, B)  :- depends-on(A, B).
depends(A, C)  :- depends-on(A, B), depends(B, C).
impacts(X, Y) :- reaches(Y, X).                  # reverse: what breaks if X changes
```

- **Proof traces.** Every derivation carries its steps — each a real fact with its `fact_id` and `source_ref` — so a caller can re-verify the whole chain.
- **Cycle-safe & depth-bounded** (BFS + a `seen` set; default max depth ~8) so evaluation always terminates on a cyclic call graph.
- **Confidence = min-of-chain** — a chain touching an LLM-distilled (<1.0) fact is visibly less than 1.0; a deterministic-only chain stays 1.0.
- **Primitives:** `reaches`, `depends`, `trace_path(src, tgt)` (shortest proof), and `impacts(x)` (the reverse call-cone: everything that reaches `x` — i.e. what a change to `x` could break).

Imagination

"Imagination" is counterfactual reasoning built on `impacts`: an `imagine_impact("X")` tool answers "what breaks if I change X?" by framing the proven impact-cone as a hypothetical; and reasoning-chain *grounded synthesis* uses the reasoner to generate genuinely structural training data (not fact-echo), which then feeds the eval-gated retrain via a de-echoed multi-hop eval.

The punchline (proven in Part V) On multi-hop questions, the neuro-symbolic layer is the *only* configuration that can answer at all — **39/40 vs 0/40** for the bare fine-tune. Multi-hop reasoning is a capability the graph *adds*, not a metric the fine-tune slightly improves.

2.6 MCP — how tools reach the model

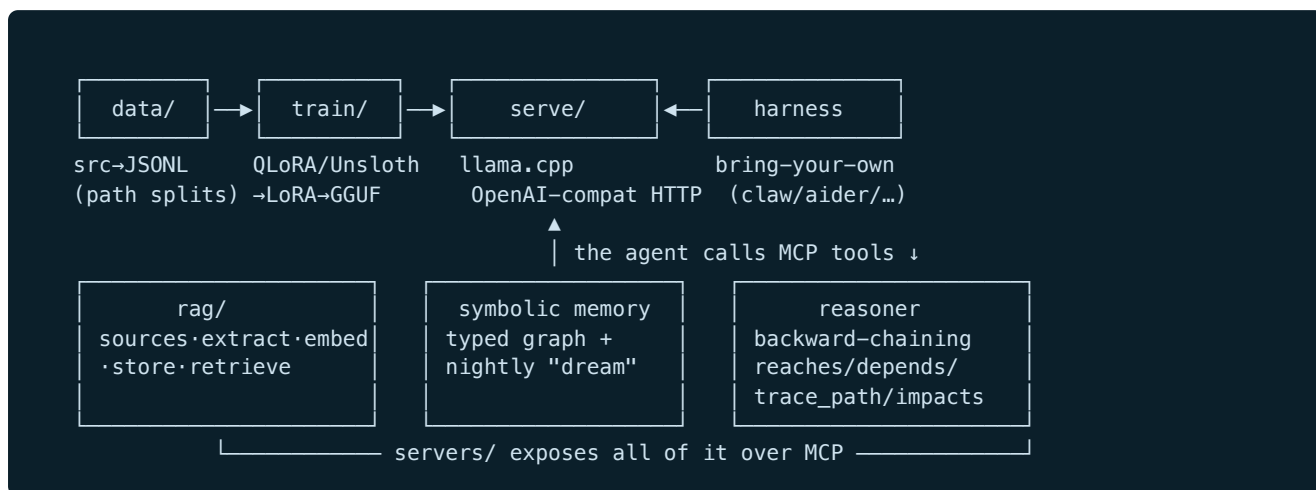
MCP (Model Context Protocol) is the standard by which the harness discovers and calls external tools. Both memories are exposed as an MCP server whose tools split cleanly:

RAG tools (fuzzy recall)	Neuro-symbolic tools (exact + reasoning)
<code>search_code</code> , <code>search_docs</code> , <code>search_changes</code> , <code>find_similar_bugs</code> , <code>get_change_diff</code>	<code>query_facts</code> , <code>explain_entity</code> , <code>trace_path</code> , <code>imagine_impact</code>

The agent decides when to call these; results are injected into its context, and a well-behaved model relays them with citations. Any OpenAI-compatible harness that speaks MCP can consume them.

Part III — Architecture & code map

3.1 The end-to-end pipeline



The four phases are locked in order and each hides behind an interface (ABC), so a phase can be reimplemented without touching the others. The lower boxes are the augmentation layers.

3.2 Package by package

The reference implementation lives under `codescribe_train/`:

Package	Purpose
<code>codescribe_train/data/</code>	Any source tree (Git working tree, Perforce workspace, or plain directory) → train/val/test JSONL. Walks files, applies a YAML config, dedups, formats (FIM / document / diff-instruction), writes deterministic path-based splits.
<code>codescribe_train/train/</code>	QLoRA via Unsloth over the JSONL splits; eval; Q4_K_M GGUF export. The eval-gated retrain is the only weight-touching path. Strictly local.
<code>codescribe_train/backends/</code> (serve)	Pluggable OpenAI-compatible inference layer. Default wraps vendored <code>llama.cpp</code> 's server; a remote backend exists only as a stub, never wired into a default.
<code>codescribe_train/harness/</code>	Adapter to whatever coding agent the operator picks; the harness is bring-your-own and treated as an opaque, low-trust subprocess.
<code>codescribe_train/rag/</code>	The RAG pipeline: <code>sources/</code> (code, docs, Perforce, Bugzilla), <code>extract/</code> , <code>embed/</code> , <code>store/</code> (sqlite-vec + FTS5 writer/retriever), and a CLI with <code>index</code> / <code>status</code> subcommands.
<code>codescribe_train/servers/</code>	MCP servers exposing both memories (search + facts + reasoning tools), plus a safe grep and doc lookup.
<code>symbolic memory + reasoner</code>	The typed knowledge graph, the nightly dream loop, and the deductive engine. Concepts + build plan are documented; this is the layer that wins the multi-hop eval.

3.3 The two data stores

Both stores are a single SQLite file each, built on the trainer and served on the serving box.

The knowledge graph

```
entities(id, kind, name, repo, path, meta{json}, first_seen, last_seen)
UNIQUE(kind,name,repo)
facts(id, subject→entities, predicate, object→entities | object_lit,
      confidence, extractor, created_at, valid_from, valid_to)    -- valid_to NULL =
currently true
provenance(id, fact→facts, source_kind, source_ref, excerpt, created_at) -- evidence, many
per fact
-- plus episodes (raw interaction stream) and lineage tables (which dream cycles fed which
weights)
-- indexes: (predicate, object, valid_to) "what calls/affects X"; (subject, predicate,
valid_to) entity traversal
```

The retrieval index

```
code_chunks + code_chunk_vectors vec0[1024] + code_chunk_fts fts5(... content='code_chunks')
-- external-content!
doc_chunks  + doc_chunk_vectors  + doc_chunk_fts
bugs        + bug_vectors        + bug_fts          -- from Bugzilla
changes     + change_vectors     + change_fts       -- from Perforce (+ chunked review
discussion)
state(key, value) -- last-indexed markers
```

Index lifecycle Build to a temp path, verify counts + *freshness* (a staleness metric — how many indexed paths no longer exist), back up, then copy to the serving box over the LAN. The retrieval server opens the file per-session, so shipping a new index needs no restart. Shipping an index must never touch the served weights.

3.4 Configuration

Per-codebase behaviour is **data, not code**: YAML under `configs/` (separate files for the data pipeline, RAG sources + embedder + chunking, the memory/dream cycle, and the retrain gate). Dependencies install *per phase* with uv extras (`uv sync --extra data|train|backends|harness|rag`); Python 3.12; each extra pulls only its stack.

Part IV — Infrastructure & operations

4.1 The two machine roles

The stack assumes two machines on a private LAN. Nothing reaches the cloud.

	The trainer	The serving box
Role	fine-tunes, dreams, builds indices (idle most of the time)	always-on; serves the GGUF, hosts the indices the harness reads
GPU	a modern ~8 GB card able to run the Unsloth QLoRA stack	any GPU that can serve a Q4_K_M 7B (older/cheaper is fine)
Notes	the <i>only</i> box that runs training; occasional, heavy	light, continuous; loopback bind + api-key

The division is fixed: heavy/occasional work (train, index, dream) on the trainer; light, always-on serving on the serving box. Indices are built on the trainer and copied to the serving box over the LAN. The served weights are replaced only by an explicit, eval-gated deploy.

4.2 Runbooks

Build / refresh the RAG index

```
python -m codescribe_train.rag index      # re-embed code + incremental Perforce/Bugzilla
python -m codescribe_train.rag status     # counts + STALENESS check (warns past a threshold)
# verify staleness ~0%, then copy the index file to the serving box over the LAN
```

Run the quality ablation (single-hop or multi-hop)

```
# two-process split so the embedder frees VRAM before the 7B loads (8 GB card):
python -m codescribe_train.train.ablation precompute --tasks evals/<suite>.json --out
/tmp/ctx.json ...
python -m codescribe_train.train.ablation run --tasks evals/<suite>.json --contexts
/tmp/ctx.json \
    --configs ft,ft_rag,ft_ns,ft_rag_ns --report logs/ablation.json
```

Retrain (the only weight-touching path)

```
python -m codescribe_train.train.retrain --config configs/retrain.yaml
# assembles real+capped-synth → trains → BEAT-OR-DISCARD gate on a frozen synth-free eval →
deploy+rollback
```

4.3 The strictly-local posture in practice

- **Egress audit:** re-run a full session under `strace -f -e trace=connect` after wiring anything new; assert zero non-loopback connects (your own LAN box excepted, named explicitly). A stubbed embedder hides the real model-load egress — drive a *real* load.
- **Vendored trust:** third-party binaries (the inference server, the harness) are SHA-pinned, never auto-updated, and verified through the binary's own `doctor` /config-dump ("loaded 0/N" means your

config was rejected wholesale).

- **Sandbox:** in production the harness runs with no network (container `--network none`).

Part V — Evaluation: does each layer earn its place?

Evaluation answers, per axis, "should I turn this layer on for this job?" — and produced the stack's headline result.

5.1 Methodology

Evaluation is a **capability × configuration matrix**, not one number. Five configurations of the same served model, differing only in what context is injected (retrieve → prepend → complete):

Config	Fine-tune	RAG ctx	NS ctx	Isolates
base	✗ stock 7B	✗	✗	the floor
ft	✓	✗	✗	"LLM only"
ft_rag	✓	✓	✗	RAG's lift
ft_ns	✓	✗	✓	the symbolic lift
ft_rag_ns	✓	✓	✓	do they compose?

Metrics: structural accuracy (fraction of a task's expected signals present in the answer, *echo-stripped*), provenance precision (fraction of `path:NN` citations pointing to a real file with ≥NN lines), citation rate, hallucination/refusal, latency.

Two threats you must always control (1) **Echo-inflated scoring** — if the scorer counts signals already in the prompt, every config looks better and deltas shrink; token-slice the generated answer only, and assert the fix is active at runtime. (2) **VRAM co-residency** — the embedder and the 7B don't co-fit on 8 GB, so precompute all retrieval/graph context in a process that *exits* before the model loads.

5.2 Single-hop results

56-task grounded structural suite (imports / defined-in / calls), fine-tune vs stock 7B:

config	structural	provenance	citation rate	hallucination	gen ms
base	0.537	1.000	0.018	0.000	2451
ft	0.590	0.867	0.232	0.000	2494
ft_rag	0.611	0.929	0.214	0.000	3238
ft_ns	0.623	0.941	0.518	0.000	2467
ft_rag_ns	0.624	0.952	0.375	0.000	3229

Every layer lifts structural accuracy a little and they compose; every augmentation improves provenance. The symbolic layer's signature single-hop win is **citation rate** — handed exact `file:line` facts, the model volunteers a checkable citation 52% of the time (vs 23% ft, 2% base), at the best precision, for ~zero extra latency. RAG adds ~740 ms.

A caveat that became a case study The first run reported ft_rag provenance at 0.73 — *below* ft — and nearly shipped "RAG hurts provenance." It was an artifact: the indexer never pruned chunks for *deleted* files, so an incremental-on-stale-base rebuild left **56% dead-path chunks**; RAG cited files that no longer existed. Fix: a prune step + a staleness detector that warns $\geq 5\%$. Corrected number: 0.929. *An ablation is only as trustworthy as the quality of the systems it compares.*

5.3 Multi-hop results — the neuro-symbolic win

40 transitive-call tasks: "does **A** transitively call **C**, through what?" The endpoints are in the prompt (they echo-strip away), so the *only* scorable signals are the **intermediate** function and a real **file:line** — exactly what a config must *chain* to produce. For the NS configs, the injected context is the reasoner's derivation.

config	structural	provenance	citation rate	named the intermediate	gen ms
ft	0.319	0.063	0.175	0 / 40	3176
ft_rag	0.244	0.000	0.000	1 / 40	2376
ft_ns	0.938	0.813	0.925	39 / 40	3179
ft_rag_ns	0.850	0.621	0.550	39 / 40	4182

1. **NS is the only config that can chain at all.** Bare FT names the connecting intermediate **0 / 40**; FT+NS **39 / 40**. A capability the others structurally lack, not a metric they slightly improve.
2. **RAG doesn't just fail multi-hop — it mildly hurts.** FT+RAG scores *below* bare FT and its citation rate collapses to 0: retrieved snippets sit around the endpoints but never contain the connecting hop, so they are noise for a structural question.
3. **NS alone beats NS+RAG here** — the derivation is already the answer; adding chunks only distracts (and adds ~1 s). On multi-hop, turn RAG *off*.

The decision this supports Read the matrix by column: **fine-tune for fluency, RAG for freshness, the symbolic layer for provenance and multi-hop reasoning.** Each pays for itself on its own axis — and RAG is actively wrong for reasoning. This is the empirical justification for building the neuro-symbolic layer at all.

Part VI — Landmines: the traps that already bit

Each cost real debugging time. Internalise them before touching the matching subsystem.

1. The eval-echo scorer

The structural scorer originally counted signals present in the *echoed prompt*, not just the generated answer. Since ~40% of expected signals appear verbatim in their prompts, **every historical gate score was inflated ~4–7 points**. Fix: token-slice the answer only. Re-gate anything measured before the fix; treat old absolute scores with suspicion.

2. Editable install shadows PYTHONPATH

An editable install resolves imports to the install's target dir, not your `PYTHONPATH` or a sibling checkout — you can "fix" code and still run the old code. Always assert the fix is live (`inspect.getsource(fn)` contains your change; `module.__file__` points where you expect).

3. External-content FTS5 delete order

Delete FTS row → vector row → content row, in that order, or you corrupt search with ghost terms (see 2.4).

4. Stale index / no prune for deleted files

The indexer replaced a file's chunks in place but never dropped chunks for files *deleted* from the codebase. Rebuilding incrementally on a stale base accumulated dead-file chunks (hit 56%), silently poisoning retrieval and any eval using RAG. Fix: a prune step + a `status` staleness metric that warns ≥5%. Run `status` after every ship.

5. Nested-checkout indexing pollution

The source walkers indexed nested working-copy/worktree checkouts, so a large fraction of the index was duplicate. Fix: exclude the agent/worktree scratch directories in the walkers.

6. NS context for multi-hop must inject the derivation, not flat facts

The ablation's NS block originally dumped flat single-file facts. For a multi-hop task that forces the *model* to chain — the very thing it can't do — so FT+NS would have looked no better than FT (a false negative). Fix: inject the reasoner's `trace_path` derivation, parsed from the *prompt* (not the answer key). *General rule: for a reasoning eval, inject the reasoner's conclusion, not the raw facts it used.*

7. Auto-generated evals need adversarial filters

The first multi-hop generator produced degenerate tasks: builtin-method "endpoints" (called everywhere, defined nowhere) and test-name-telegraphed intermediates. Fix: require endpoint + intermediate to be `defined-in` the codebase; reject telegraphed intermediates; prefer mid-frequency callees. Read your generated tasks and ask "how could this be answered the wrong way?"

8. Vendored-binary config schema

A stray key made a vendored binary reject its whole config ("loaded 0/N"), silently disabling a safety gate. When rendering config for a program you didn't write: confirm the discovery path, the schema (does it

strict-reject unknown keys?), and end-to-end via the binary's own `doctor` /dump. A config read from source is necessary but not sufficient.

Meta-lesson Prefer **evidence over assertion** everywhere: a reproduced bug over a suspected one, a verified swap over an assumed one, the binary's own report over a schema you read, a runtime assertion that your fixed code is the code running. Almost every landmine bit because a plausible-looking assumption went unchecked.

Part VII — Current state & roadmap

Built and verified

- The four phases: data → train → serve → harness. End-to-end verified by egress audit (zero non-loopback connects).
- RAG: sqlite-vec + FTS5 hybrid with RRF over code/docs/Bugzilla/Perforce; MCP tools; a staleness detector + prune; clean index shipped to the serving box.
- Neuro-symbolic: the graph with provenance + temporal supersession; the dream loop; the deductive reasoner; imagination tools (`trace_path` , `imagine_impact`).
- Eval-gated retrain with anti-collapse gates; a first real promotion of a fine-tune over the prior one.
- Evaluation: the ablation harness + methodology + single-hop and multi-hop results.

The next major build

- **Grounded synthesis.** Use the reasoner to generate genuinely structural (multi-hop, cross-file) training data — the "fundamentally different" synth that should actually teach structure — and feed it to the eval gate via a de-echoed multi-hop eval. This closes the loop: reasoning → better training data → better fine-tune → repeat, gated so it can't collapse.
- **Semantic entity lookup** (embedding the graph's entities) — deferred.

Where to point yourself first

1. Read the overview and architecture concept docs (the curated entry points).
2. Skim the reasoner end-to-end — it's small and the intellectual core.
3. Run the test suite (GPU-free) to see the retrieve→inject→score loop concretely.
4. Reproduce a small ablation to watch each layer's contribution.
5. Read the lessons/landmines with full file:line context before touching a subsystem.

Appendix A — Glossary

Term	Meaning
QLoRA	Quantized Low-Rank Adaptation: fine-tune by learning small adapters over a 4-bit-quantized frozen base. Fits a 7B on 8 GB.
LoRA adapter	the small trained delta (tens of MB) added to the frozen base; swappable across generations.
GGUF / Q4_K_M	the file format llama.cpp serves / its 4-bit quantization scheme.
FIM	Fill-In-the-Middle: a training format giving the model prefix+suffix to predict the middle.
RAG	Retrieval-Augmented Generation: retrieve external text at inference and prepend it to the prompt.
Embedding	a dense vector (1024-d here, from bge-large) placing semantically similar texts near each other.
sqlite-vec / vec0	SQLite extension + virtual-table type for storing vectors and doing KNN inside SQLite.
FTS5	SQLite full-text search (inverted index, BM25, stemming); the lexical half of hybrid retrieval.
RRF	Reciprocal Rank Fusion: merge ranked lists by $\sum 1/(k+\text{rank})$; scale-agnostic.
Neuro-symbolic (NS)	combining the neural model with a symbolic knowledge graph + logical reasoner.
Entity / Fact / Provenance	graph node / (subject-predicate-object) edge / the evidence (source_ref) backing a fact.
Backbone fact	a fact from a deterministic AST/regex extractor (confidence 1.0), vs. an LLM-distilled fuzzy fact (<1.0).
Backward chaining	proving a goal by recursively proving its sub-goals; how the reasoner derives transitive facts.
Derivation / proof trace	a derived fact plus the ordered real facts (with file:line) that prove it.
impacts / imagine_impact	the reverse call-cone of X (everything that reaches X) = "what breaks if I change X".
Dream loop	the nightly ingest→extract→reflect→synthesize→[retrain] memory-consolidation cycle.
Model collapse	quality decay from training on model-generated data recursively; why retrain is eval-gated.
MCP	Model Context Protocol: how the agent discovers and calls tools (the memories are an MCP server).
Echo-stripping	scoring only signals the model <i>generated</i> , not ones echoed from the prompt.
Ablation	turning layers on/off to measure each one's contribution per metric.

Appendix B — Package & command index

Reference-implementation layout

codescribe_train/data/	source tree → JSONL splits (FIM/document/diff)
codescribe_train/train/ablation	QLoRA/Unsloth training, eval, GGUF export, eval-gated retrain,
codescribe_train/backends/	inference backends (default: vendored llama.cpp server)
codescribe_train/harness/	bring-your-own coding-agent adapter
codescribe_train/rag/`index`/`status` CLI	sqlite-vec + FTS5 index: sources/extract/embed/store/retrieve,
codescribe_train/servers/configs/	the MCP servers (search + facts + reasoning tools)
evals/	per-codebase behaviour (data, rag, memory/dream, retrain)
docs/	the structural + multi-hop task suites
	the concept + runbook + lessons documentation tree

Commands

uv sync --extra rag train data harness backends	per-phase deps (Python 3.12)
python -m pytest tests/ -q	the test suite (GPU-free)
python -m codescribe_train.rag index status	build / check the RAG index
python -m codescribe_train.train.ablation ...	the quality matrix
strace -f -e trace=connect ...	the egress audit (expect zero non-
loopback)	

This handbook is the reading-order narrative that threads the concept docs together for someone starting cold. It names no specific codebase, host, or operator — it is the teachable version of the stack. Regenerate it whenever the design moves enough that the details drift.